

# How Coding Agents *Fail* Their Users

A Large-Scale Analysis of Developer-Agent Misalignment in 20,574 Real-World Sessions

Ningzhi Tang<sup>1</sup>, Chaoran Chen<sup>1</sup>, Gelei Xu<sup>1</sup>, Yiyu Shi<sup>1</sup>, Yu Huang<sup>2</sup>, Collin McMillan<sup>1</sup>, Tao Dong<sup>3</sup>, Toby Jia-Jun Li<sup>1</sup>

<sup>1</sup>University of Notre Dame · <sup>2</sup>Vanderbilt University · <sup>3</sup>Google · ntang@nd.edu, toby.j.li@nd.edu

I'm graduating in Dec 2026 and on the job market for full-time industry roles.

Agent evaluation · coding agents · agent customization  
developer tools and experience · human-AI interaction

✉ ntang@nd.edu

🌐 www.nztang.com

🔗 TTangNingzhi

📄 in/ningzhi-tang

## 01 · MOTIVATION

### Why real sessions?

Existing studies of coding-agent failures replay benchmark trajectories. We study misalignment as developers experience it in real-world sessions.

#### DEFINITION

*Misalignment is a breakdown in developer-agent collaboration made visible by the developer's correction or pushback.*

**INSTRUCTION** what developers explicitly ask for

**INTENTION** what they actually want

## 02 · DATA

### 20,574 real sessions, two sources

BUILT ON OUR PRIOR WORK · ASE 2026

#### Programming by Chat

Characterized developers' prompt behavioral intents, session structures, and interaction dynamics.



#### SPECTORY

14,789

1,441 repos

IDE + CLI · Sep 2024–Apr 2026

An IDE extension writes session as Markdown to  
.specstory/history/ and commits it to the repo.

#### ENTIRE.IO

5,785

198 repos

CLI · Jan–Apr 2026 · via SWE-chat

Git hooks save sessions as JSONL to branch  
entire/checkpoints/v1 and sync them on push.

20,574 SESSIONS · 1,639 REPOS · BY AGENT

| IDE            | 12,231 | CLI         | 8,343 |
|----------------|--------|-------------|-------|
| Cursor         | 3,234  | Claude Code | 6,648 |
| GitHub Copilot | 366    | OpenCode    | 624   |
| Unknown *      | 8,631  | Codex       | 517   |
|                |        | Gemini CLI  | 39    |
|                |        | Cursor CLI  | 32    |
|                |        | Unknown *   | 483   |

\* agent identity missing from early SpecStory exports

## 03 · METHOD

### A three-stage LLM pipeline

ALL STAGES · GPT-5.4

#### ① EXTRACT

29,896

candidate episodes

The LLM scans each full session and grounds every episode in specific log turns.

EACH RECORD 

|      |      |             |          |
|------|------|-------------|----------|
| name | goal | description | evidence |
|------|------|-------------|----------|

#### ② VALIDATE

16,118

kept (53.9%)

A second pass drops unsupported claims.

**0.93** precision **1.77/2** mean recall rating

**42.3%** Normative prior bias

flags behavior the developer never objected to

**57.7%** Observational blind spots

relies on context not in the log

#### ③ ANNOTATE

4 axes

per episode

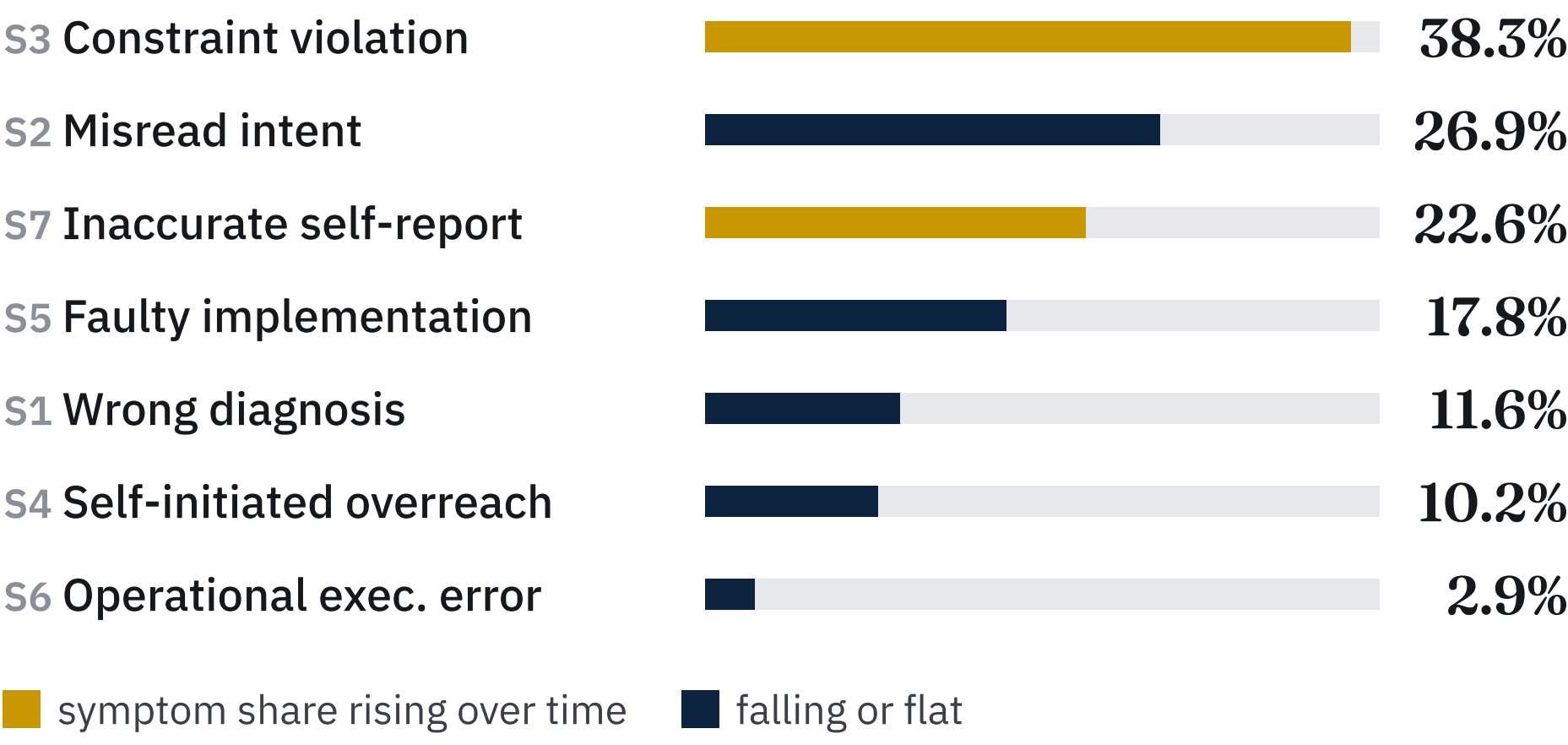
Symptom · cause · outcome · resolution.

**0.78** human agreement **0.82** judge accuracy

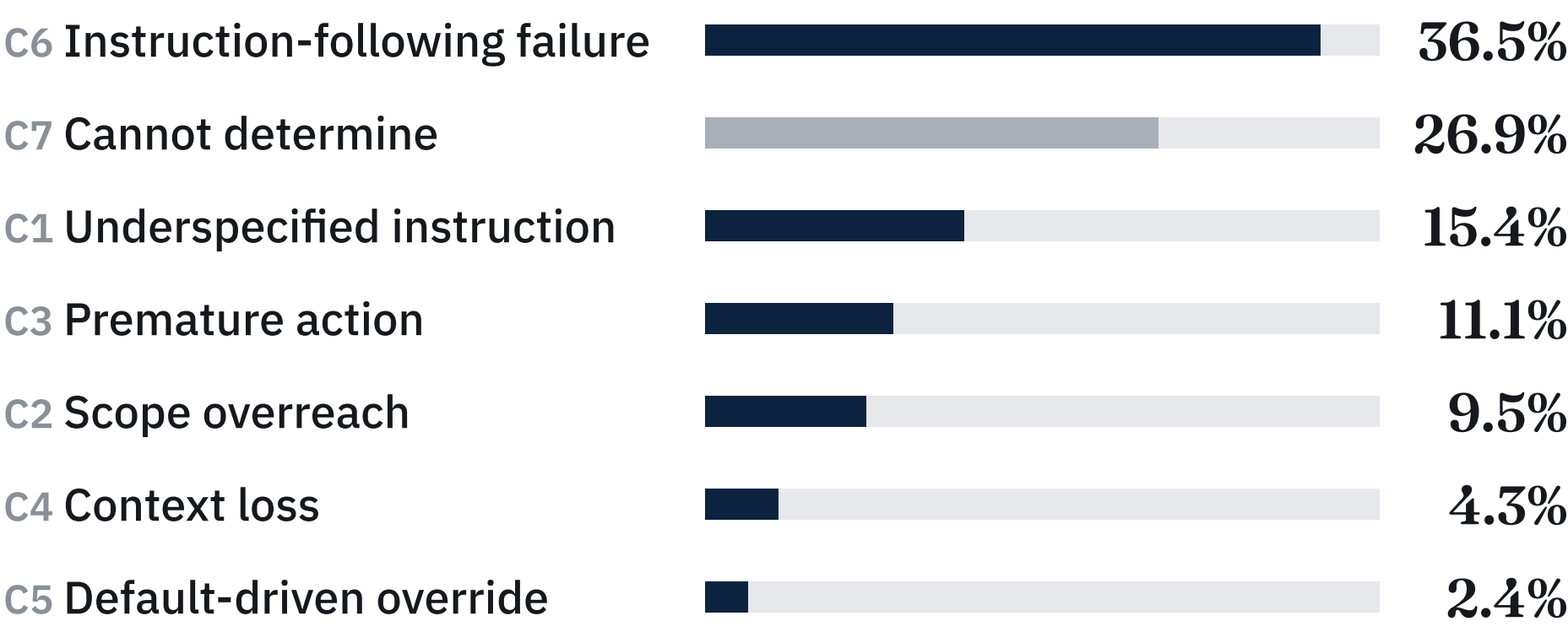
## RQ1 · FORMS & CAUSES

### What goes wrong, and why

SYMPTOMS · WHAT GOES WRONG



CAUSES · WHY

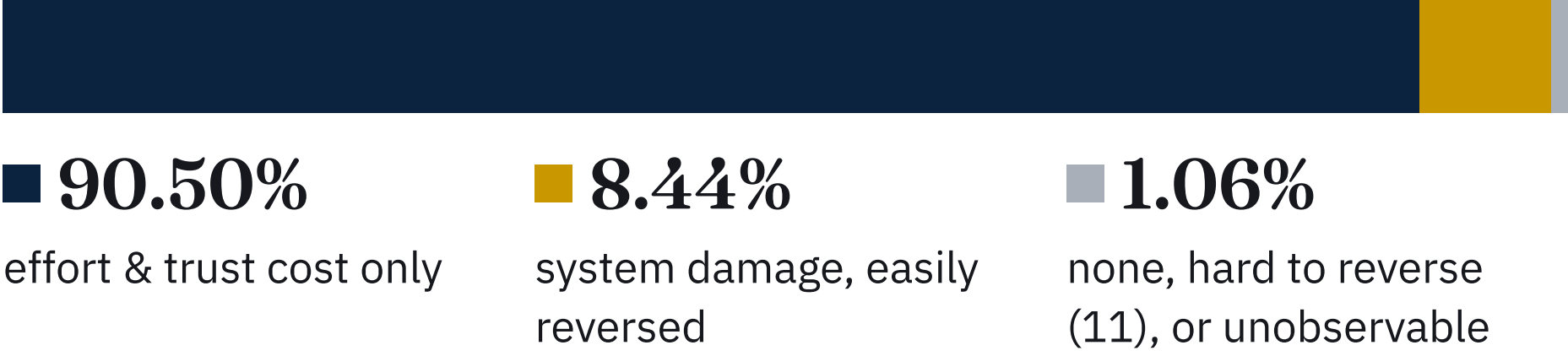


## RQ2 · OUTCOMES & RESOLUTION

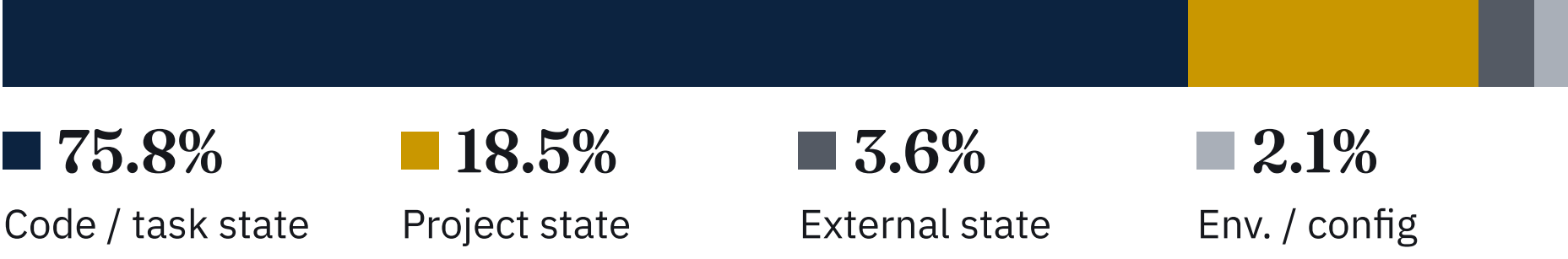
### Cheap damage, costly attention

Most episodes cause no lasting system damage. The cost falls on the developer's time and trust.

DAMAGE SEVERITY · ALL 16,118 EPISODES



WHERE DAMAGE LANDS · 1,372 EPISODES



WHO FIXES IT? · 1,504 EPISODES WITH A VISIBLE FIX



#### EXAMPLE

### One episode, from log to labels

EXTRACTED RECORD

**name** Changed Slide Orientation on an Unverified Assumption  
**goal** intention  
**description** The developer asked why slide 2 was landscape; the agent changed it to 4:3 portrait instead of explaining.  
**evidence** T7 user "why is slide 2 showing landscape"  
T8 agent "Fixed by explicitly setting the aspect ratio to 43 (4:3 portrait)..."  
T9 user "i want the whole presentation 16:9 landscape"

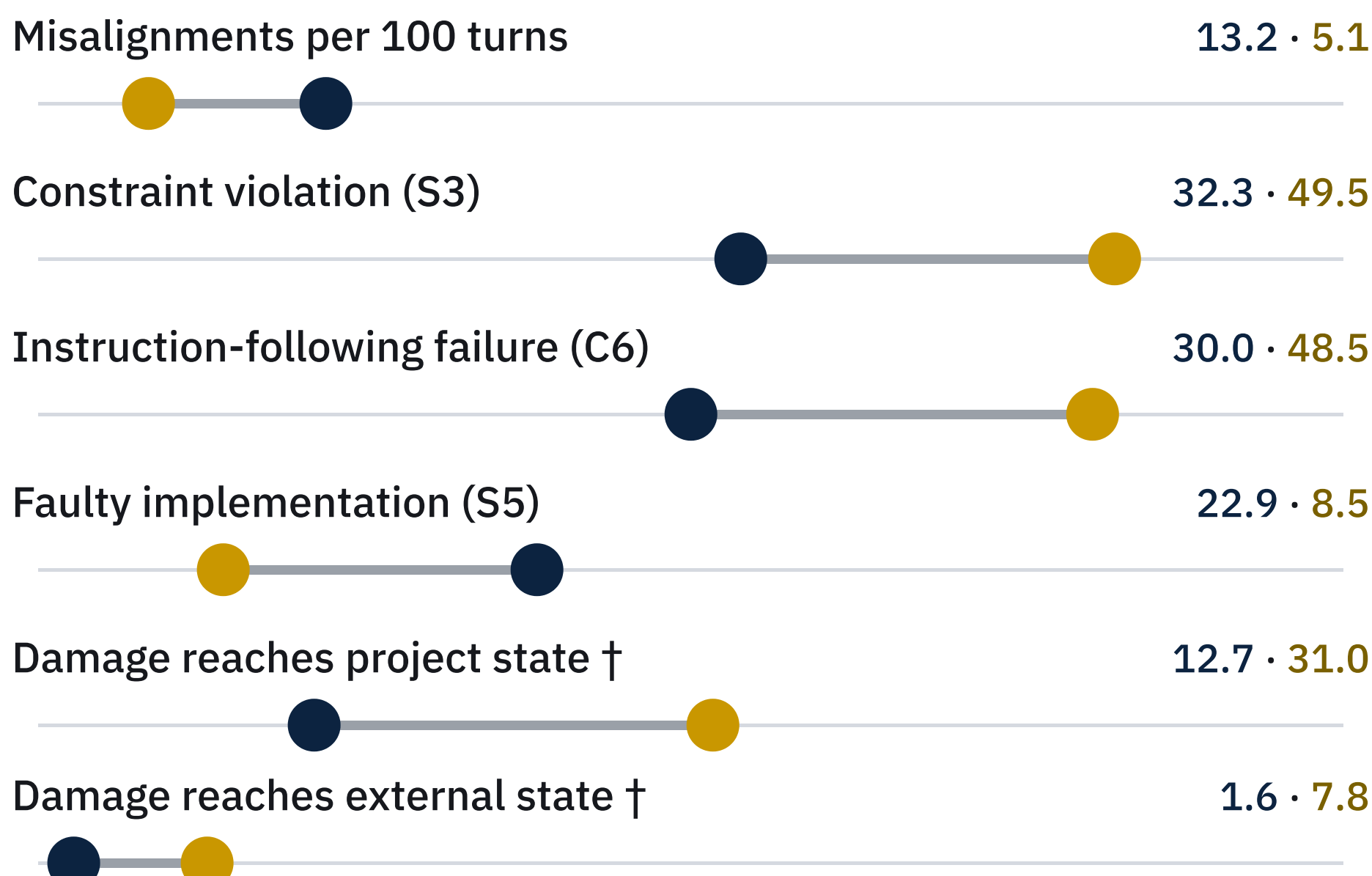
S4 OVERREACH S2 MISREAD INTENT C2 SCOPE OVERREACH

## RQ3 · IDE VS. CLI

### CLI agents break rules; IDE agents break code

Interaction shapes attention: IDE developers watch the code; CLI developers hand over control and watch the rules.

● IDE · median 3 turns/session ● CLI · median 5 turns/session



## RQ4 · STRUCTURE & TIME

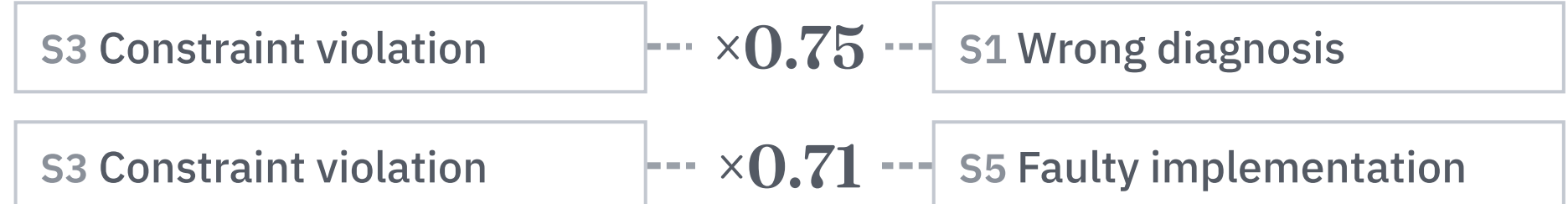
### Misalignment co-occurs, persists, and shifts

WITHIN A SESSION · CO-OCCURRENCE LIFT

Above chance



Below chance



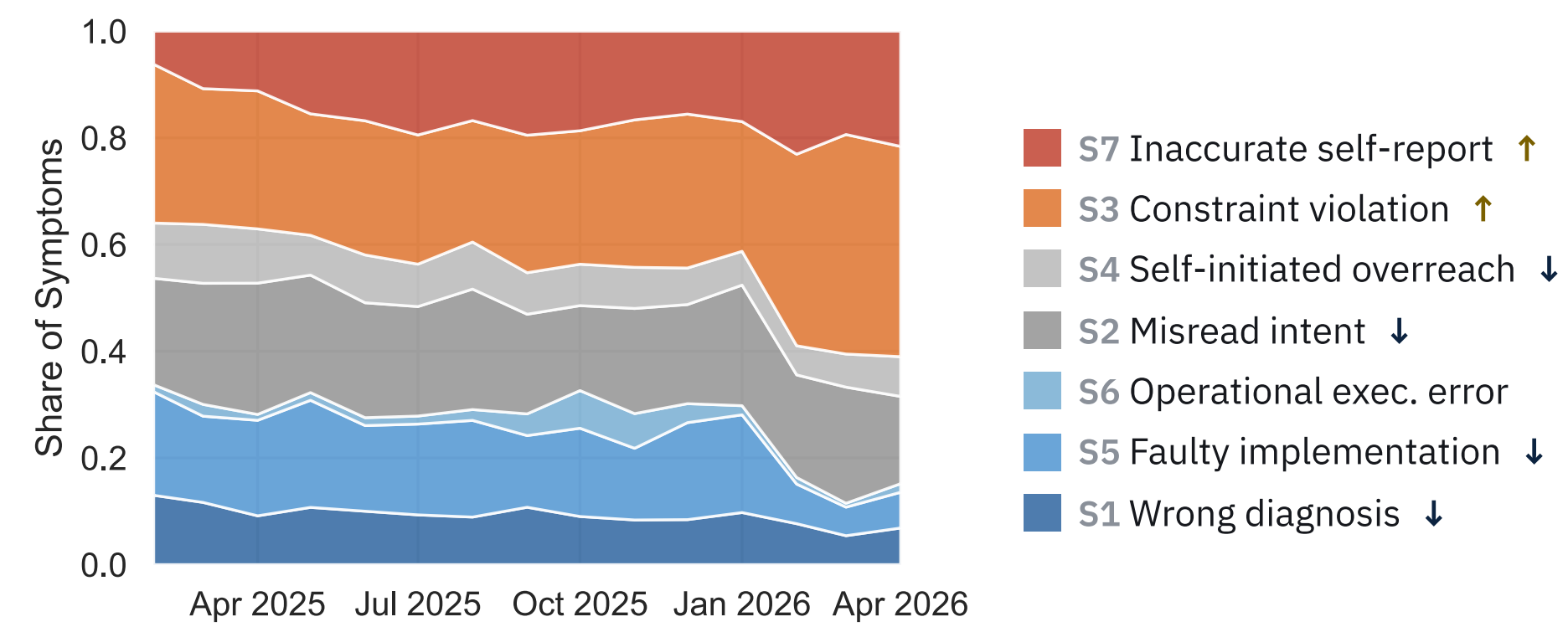
lift = observed ÷ expected co-occurrence; ×1 = chance

ACROSS SESSIONS · SAME REPO

+54% higher probability of misalignment in the next session (0.519 after a misaligned session vs. 0.336 otherwise)

OVER TIME · FEB 2025 → APR 2026

Misalignment per user turn declines ≈ 0.008 per month.



### Explore the data



INTERACTIVE DASHBOARD  
coding-agent-misalignment.netlify.app



CODE, LABELS & NOTEBOOKS  
github.com/ND-SaNDwichLAB/coding-agent-misalignment

## TAKEAWAY 1

### Interaction-level failures are the new gap

Code-level symptoms decline in share, while constraint violations and inaccurate self-reporting rise.

## TAKEAWAY 2

### Today's safety rests on the developer

Most visible fixes require real-time developer pushback. That oversight may not scale to longer-running background agents.

## TAKEAWAY 3

### Session logs are a behavioral signal

The pipeline could turn live-session misalignment into evaluation cases and training signals for agent teams.